IN-ORDER TRAVERSAL

Code

```
void in_order_traversal(Node* node)
{
    if(node == NULL){
        return;
    }
    in_order_traversal(node->left);
    cout << node->value << " ";
    in_order_traversal(node->right);
}
```

```
int main()
{
    Node* root = new Node(3);
    Node* node1 = new Node(5);
    Node* node2 = new Node(7);
    Node* node3 = new Node(4);
    root->left = node1;
    root->right = node2;
    node1->left = node3;

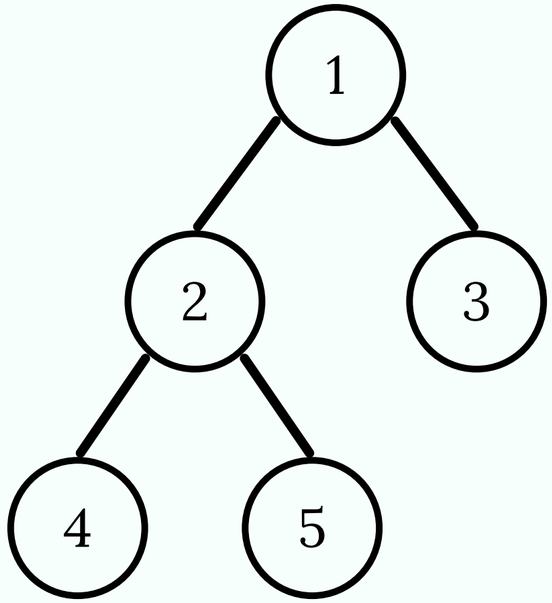
    in_order_traversal(root);
}
```

- Visit left sub-tree recursively first
- Visit parent next
- Visit right sub-tree recursively last

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

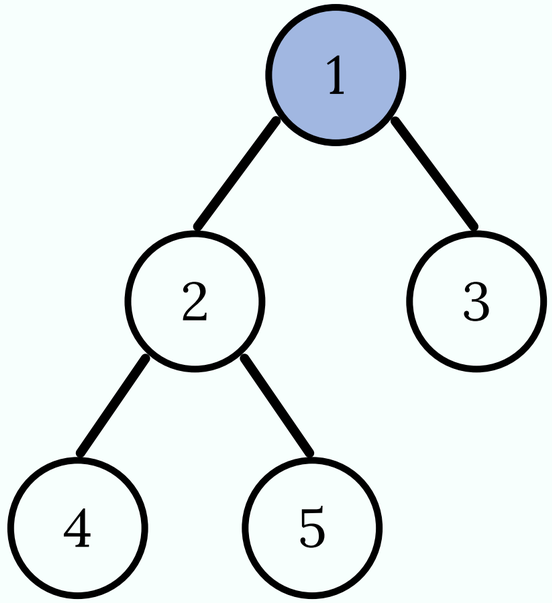


In-Order Traversal Sequence:

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	
3	
4	
5	
6	
7	
8	
9	
10	

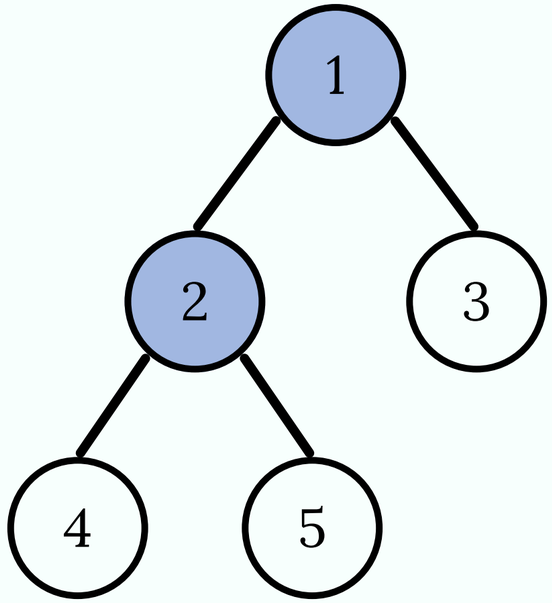


In-Order Traversal Sequence:

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	
4	
5	
6	
7	
8	
9	
10	

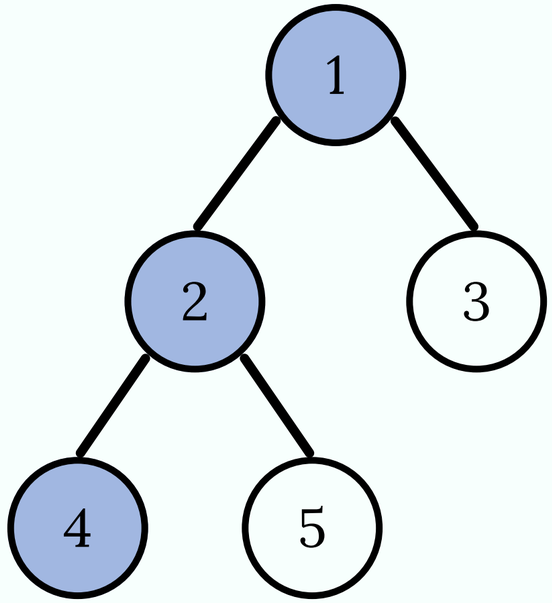


In-Order Traversal Sequence:

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	
5	
6	
7	
8	
9	
10	

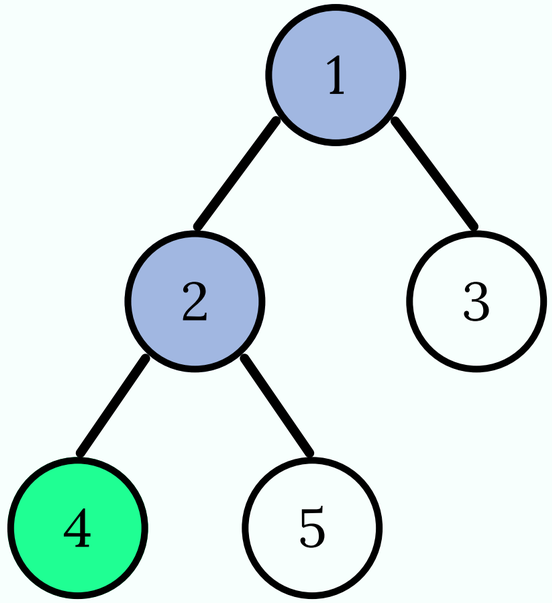


In-Order Traversal Sequence:

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	
5	
6	
7	
8	
9	
10	



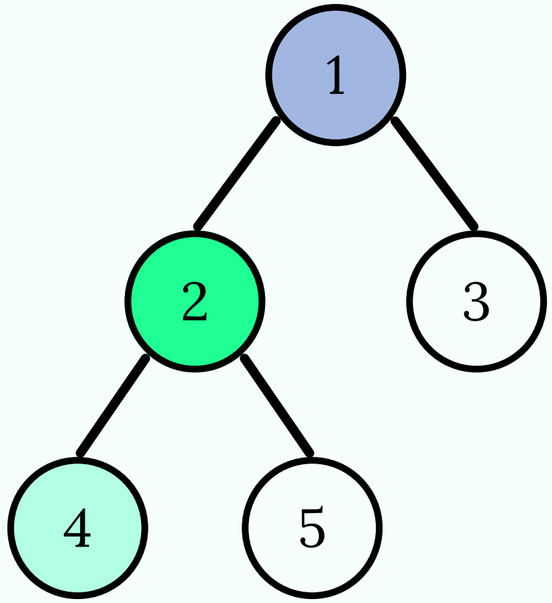
In-Order Traversal Sequence:

4

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	
6	
7	
8	
9	
10	



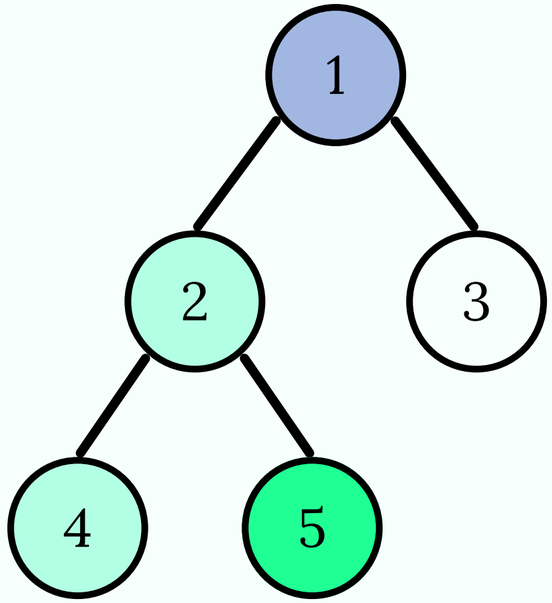
In-Order Traversal Sequence:

4 2

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	in_order_traversal (Node(5))
6	
7	
8	
9	
10	



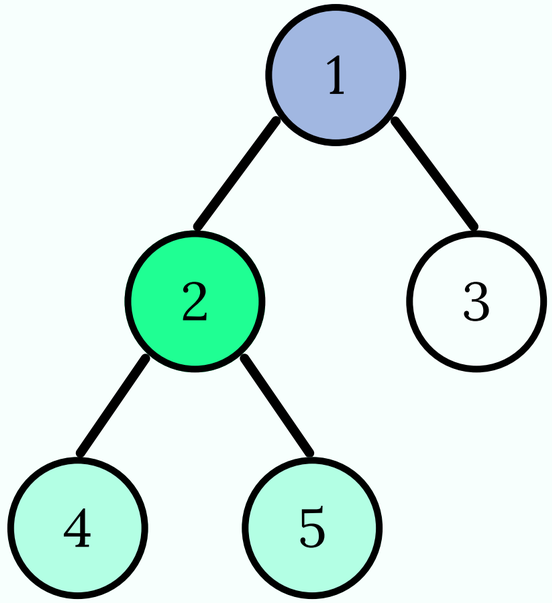
In-Order Traversal Sequence:

4 2 5

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	in_order_traversal (Node(5))
6	return to in_order_traversal (Node(2))
7	
8	
9	
10	



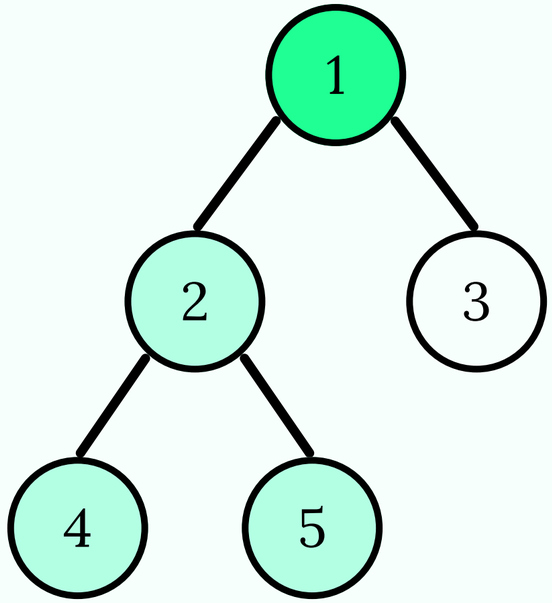
In-Order Traversal Sequence:

4 2 5

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	in_order_traversal (Node(5))
6	return to in_order_traversal (Node(2))
7	return to in_order_traversal (Node(1))
8	
9	
10	



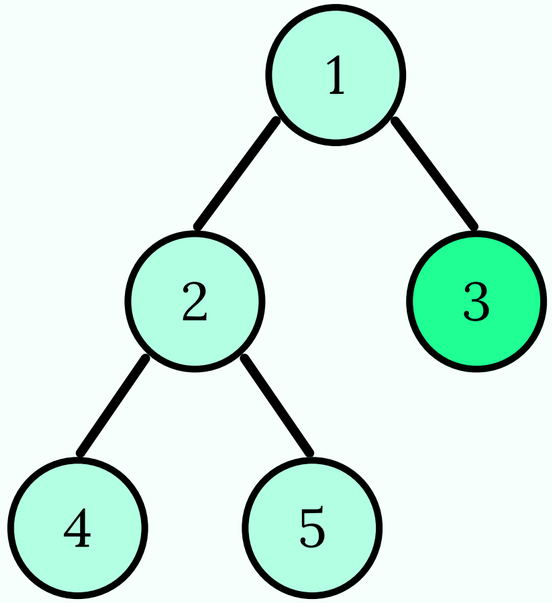
In-Order Traversal Sequence:

4 2 5 1

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	in_order_traversal (Node(5))
6	return to in_order_traversal (Node(2))
7	return to in_order_traversal (Node(1))
8	in_order_traversal (Node(3))
9	
10	



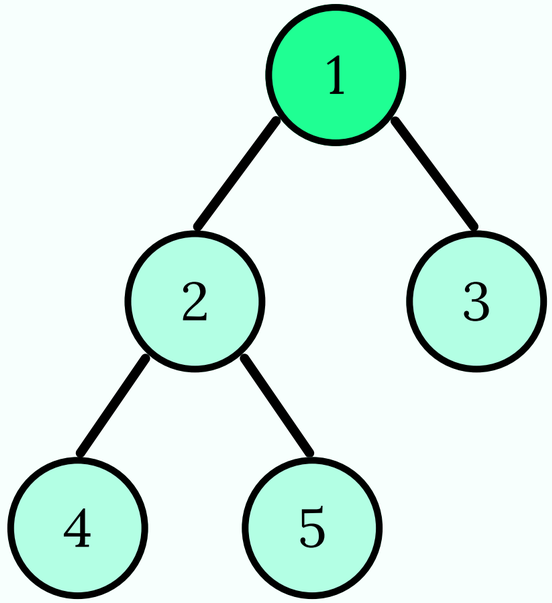
In-Order Traversal Sequence:

4 2 5 1 3

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	in_order_traversal (Node(5))
6	return to in_order_traversal (Node(2))
7	return to in_order_traversal (Node(1))
8	in_order_traversal (Node(3))
9	return to in_order_traversal (Node(1))
10	



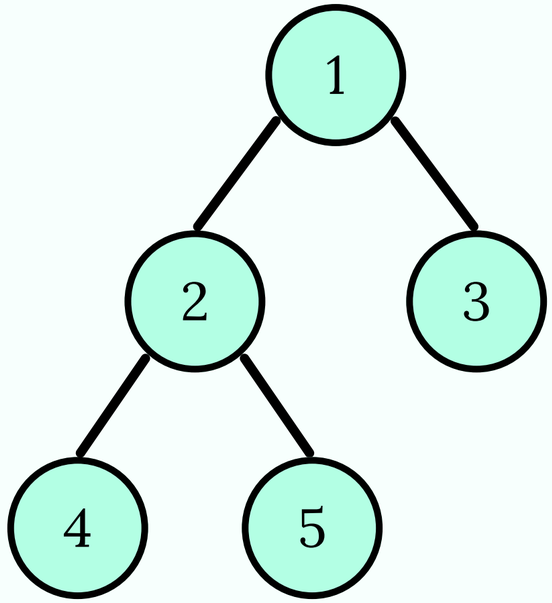
In-Order Traversal Sequence:

4 2 5 1 3

IN-ORDER TRAVERSAL

In-Order Traversal visits nodes in the sequence: **Left → Parent → Right**

Step	Function Calls
1	in_order_traversal (Node(1))
2	in_order_traversal (Node(2))
3	in_order_traversal (Node(4))
4	return to in_order_traversal (Node(2))
5	in_order_traversal (Node(5))
6	return to in_order_traversal (Node(2))
7	return to in_order_traversal (Node(1))
8	in_order_traversal (Node(3))
9	return to in_order_traversal (Node(1))
10	return to main

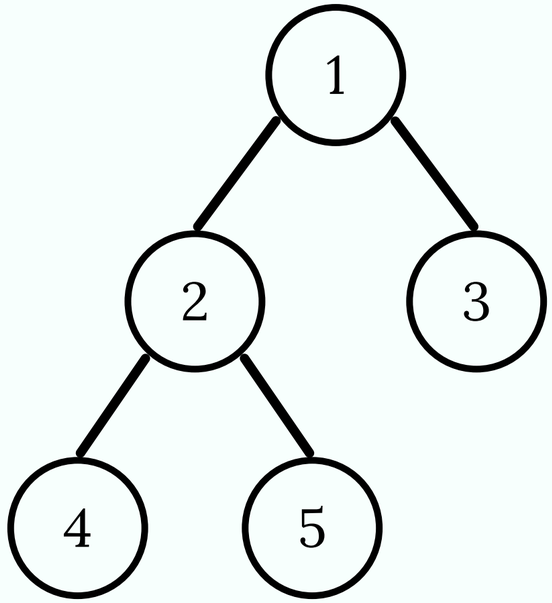


In-Order Traversal Sequence:

4 2 5 1 3

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**



POST-ORDER TRAVERSAL

Code

```
void post_order_traversal(Node* node)
{
    if(node == NULL){
        return;
    }
    post_order_traversal(node->left);
    post_order_traversal(node->right);
    cout << node->value << " ";
}
```

```
int main()
{
    Node* root = new Node(3);
    Node* node1 = new Node(5);
    Node* node2 = new Node(7);
    Node* node3 = new Node(4);
    root->left = node1;
    root->right = node2;
    node1->left = node3;

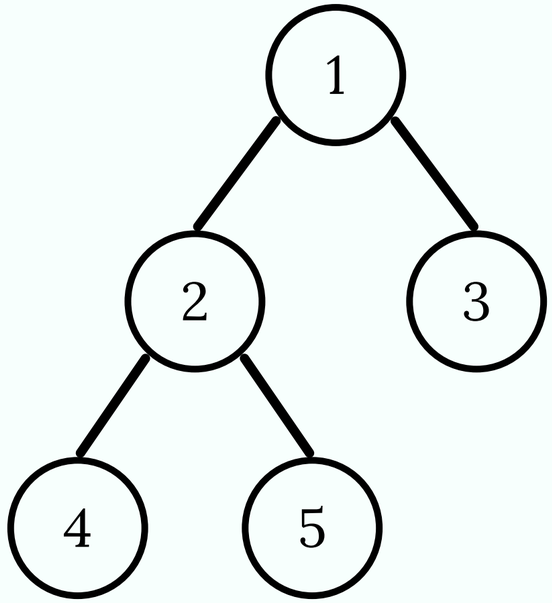
    post_order_traversal(root);
}
```

- Visit left sub-tree recursively first
- Visit right sub-tree recursively next
- Visit parent last

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

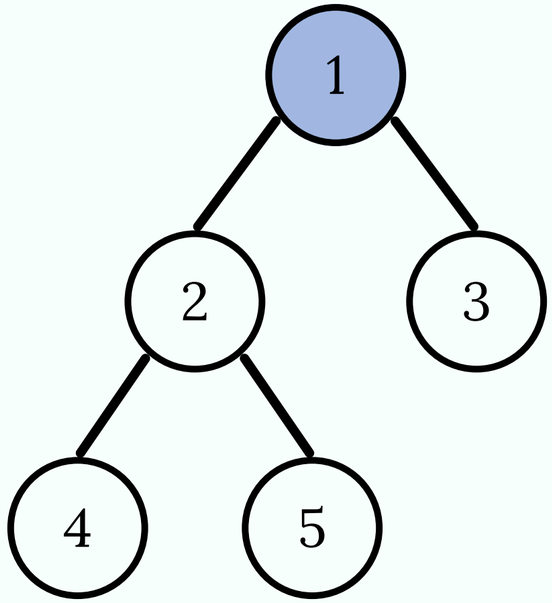


Post-Order Traversal Sequence:

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	
3	
4	
5	
6	
7	
8	
9	
10	

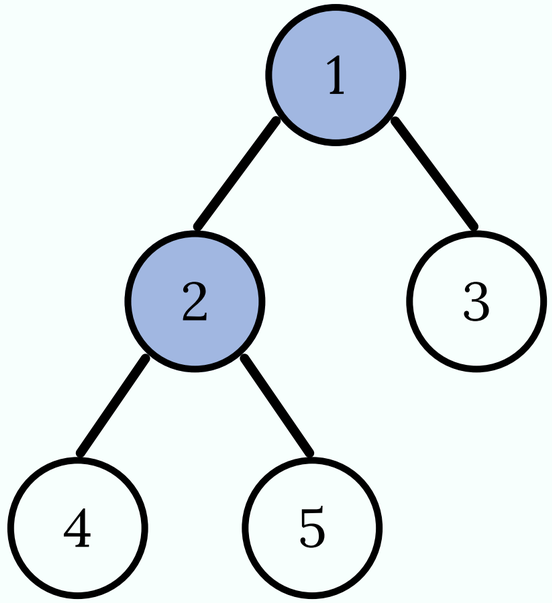


Post-Order Traversal Sequence:

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	
4	
5	
6	
7	
8	
9	
10	

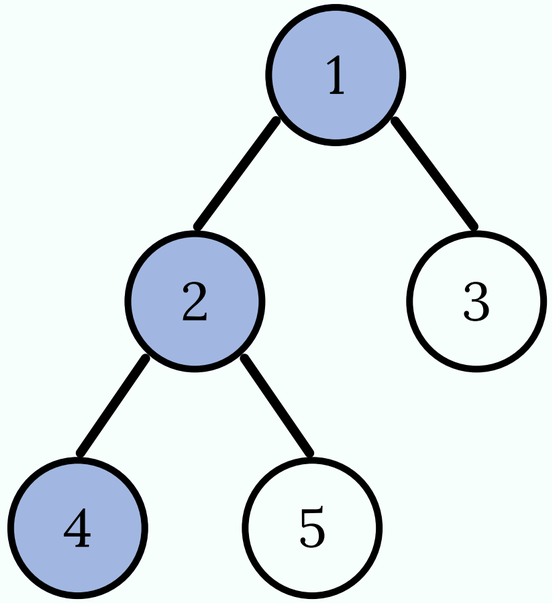


Post-Order Traversal Sequence:

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	
5	
6	
7	
8	
9	
10	

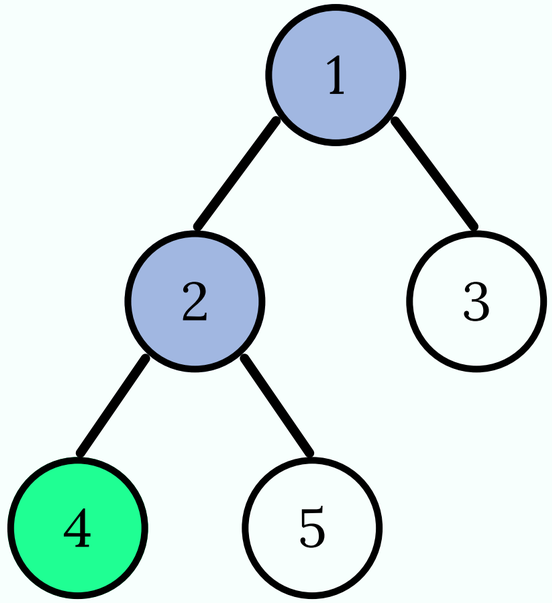


Post-Order Traversal Sequence:

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	
5	
6	
7	
8	
9	
10	



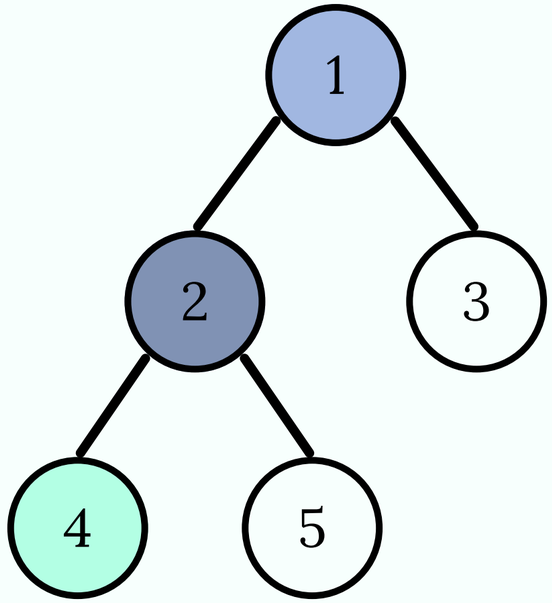
Post-Order Traversal Sequence:

4

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	
6	
7	
8	
9	
10	



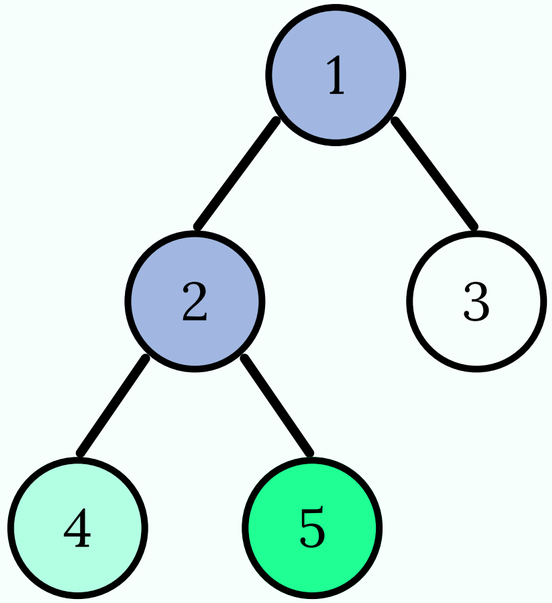
Post-Order Traversal Sequence:

4

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	post_order_traversal (Node(5))
6	
7	
8	
9	
10	



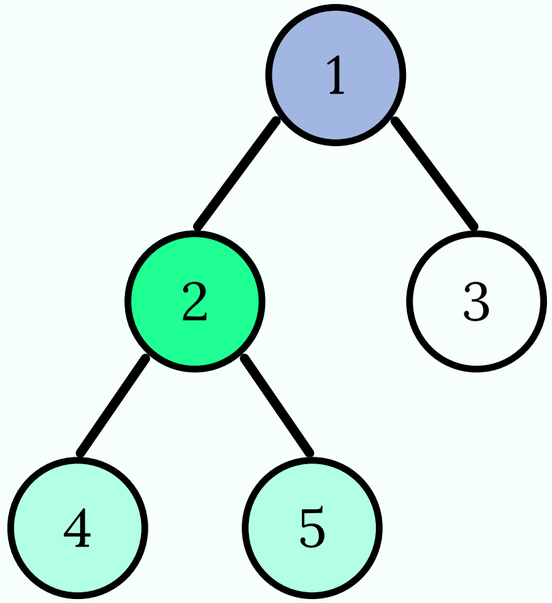
Post-Order Traversal Sequence:

4 5

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	post_order_traversal (Node(5))
6	return to post_order_traversal (Node(2))
7	
8	
9	
10	



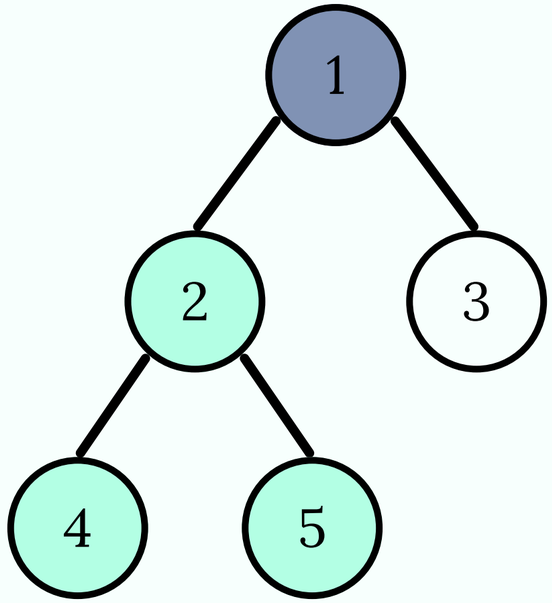
Post-Order Traversal Sequence:

4 5 2

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	post_order_traversal (Node(5))
6	return to post_order_traversal (Node(2))
7	return to post_order_traversal (Node(1))
8	
9	
10	



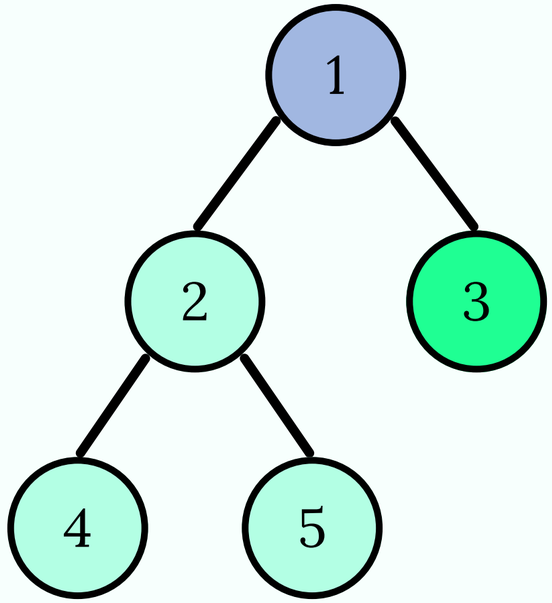
Post-Order Traversal Sequence:

4 5 2

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	post_order_traversal (Node(5))
6	return to post_order_traversal (Node(2))
7	return to post_order_traversal (Node(1))
8	post_order_traversal (Node(3))
9	
10	



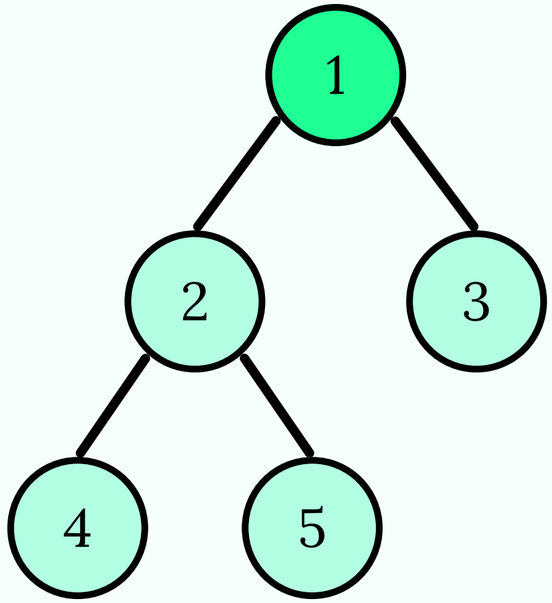
Post-Order Traversal Sequence:

4 5 2 3

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	post_order_traversal (Node(5))
6	return to post_order_traversal (Node(2))
7	return to post_order_traversal (Node(1))
8	post_order_traversal (Node(3))
9	return to post_order_traversal (Node(1))
10	



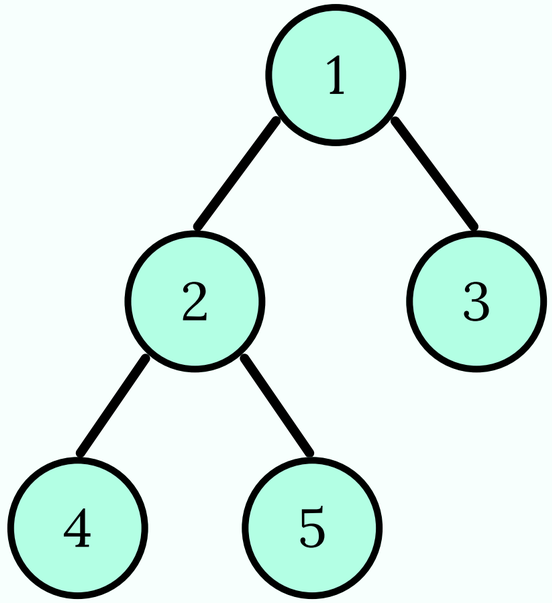
Post-Order Traversal Sequence:

4 5 2 3 1

POST-ORDER TRAVERSAL

Post-Order Traversal visits nodes in the sequence: **Left → Right → Parent**

Step	Function Calls
1	post_order_traversal (Node(1))
2	post_order_traversal (Node(2))
3	post_order_traversal (Node(4))
4	return to post_order_traversal (Node(2))
5	post_order_traversal (Node(5))
6	return to post_order_traversal (Node(2))
7	return to post_order_traversal (Node(1))
8	post_order_traversal (Node(3))
9	return to post_order_traversal (Node(1))
10	return to main()

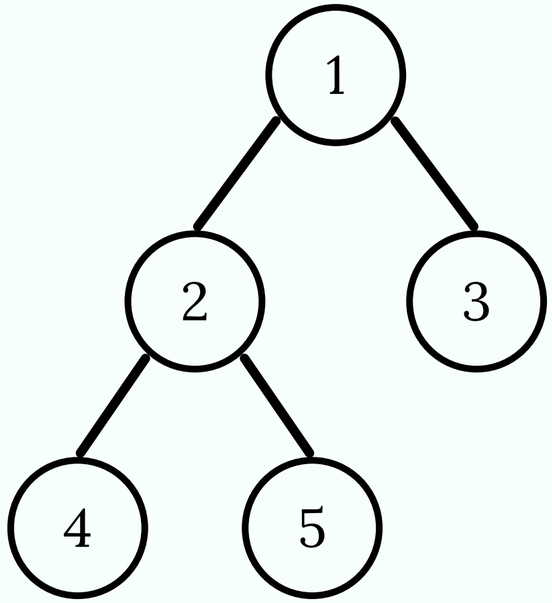


Post-Order Traversal Sequence:

4 5 2 3 1

LEVEL-ORDER TRAVERSAL

Level-Order Traversal visits nodes level-by-level



LEVEL-ORDER TRAVERSAL

Code (Same algorithmic structure as BFS)

```
void level_order_traversal(Node* node)
{
    queue<Node*> q;
    q.push(node);
    while(!q.empty())
    {
        Node* current_node = q.front();
        q.pop();
        cout << current_node->value << " ";
        if(current_node->left != NULL){
            q.push(current_node->left);
        }
        if(current_node->right != NULL){
            q.push(current_node->right);
        }
    }
}
```

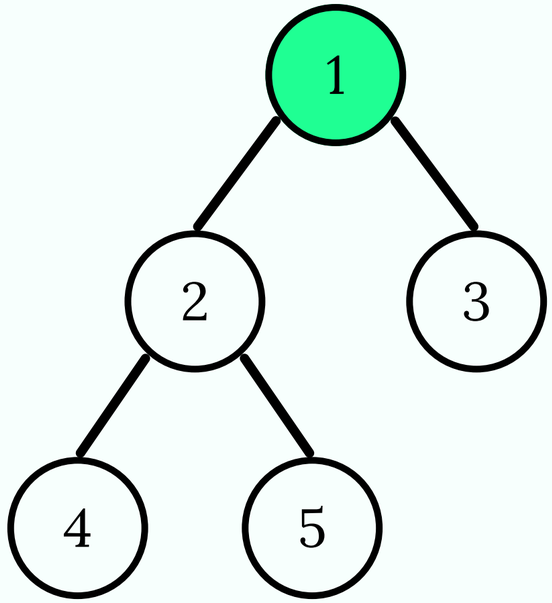
```
int main()
{
    Node* root = new Node(3);
    Node* node1 = new Node(5);
    Node* node2 = new Node(7);
    Node* node3 = new Node(4);
    root->left = node1;
    root->right = node2;
    node1->left = node3;

    level_order_traversal(root);
}
```

- Take a queue
- Push start node (root) into queue
- While the queue is not empty:
  - Pop a node from the queue
  - Process the node
  - If left-child is not NULL:
    - Push left-child into queue
  - If right-child is not NULL:
    - Push right-child into queue

LEVEL-ORDER TRAVERSAL

Level-Order Traversal visits nodes level-by-level

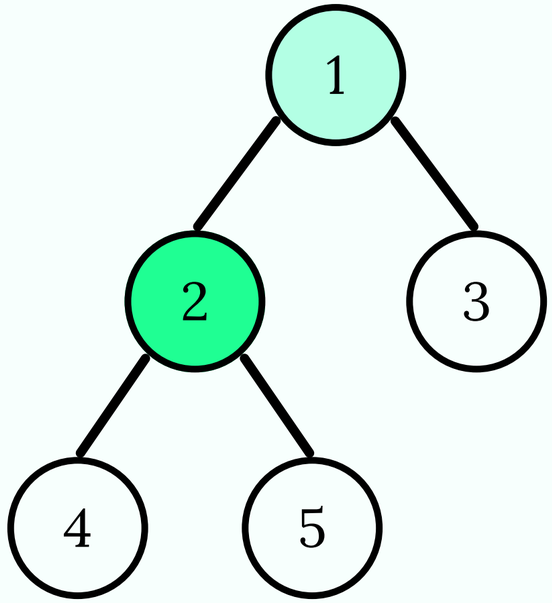


Level-Order Traversal Sequence:

1

LEVEL-ORDER TRAVERSAL

Level-Order Traversal visits nodes level-by-level

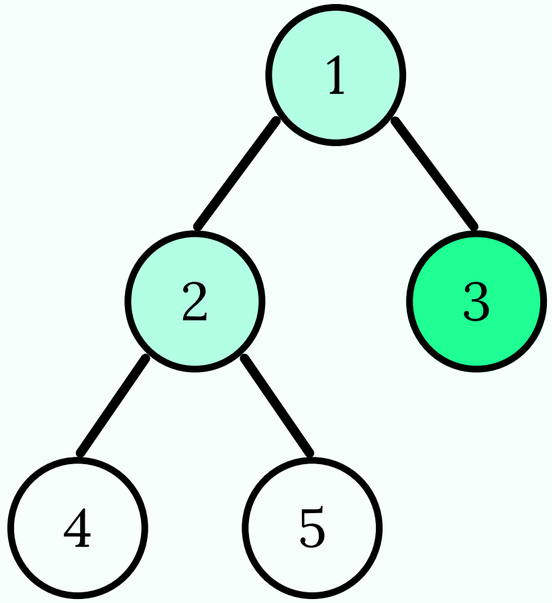


Level-Order Traversal Sequence:

1 2

LEVEL-ORDER TRAVERSAL

Level-Order Traversal visits nodes level-by-level

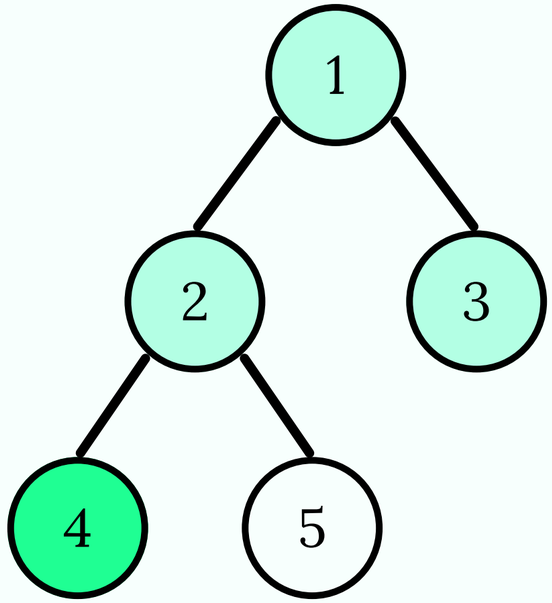


Level-Order Traversal Sequence:

1 2 3

LEVEL-ORDER TRAVERSAL

Level-Order Traversal visits nodes level-by-level

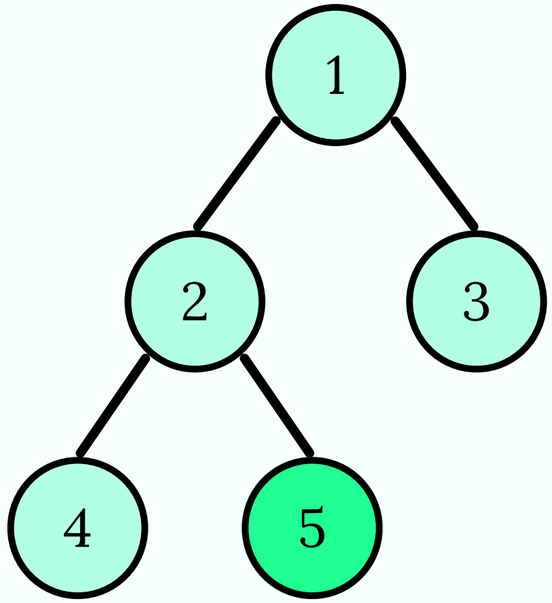


Level-Order Traversal Sequence:

1 2 3 4

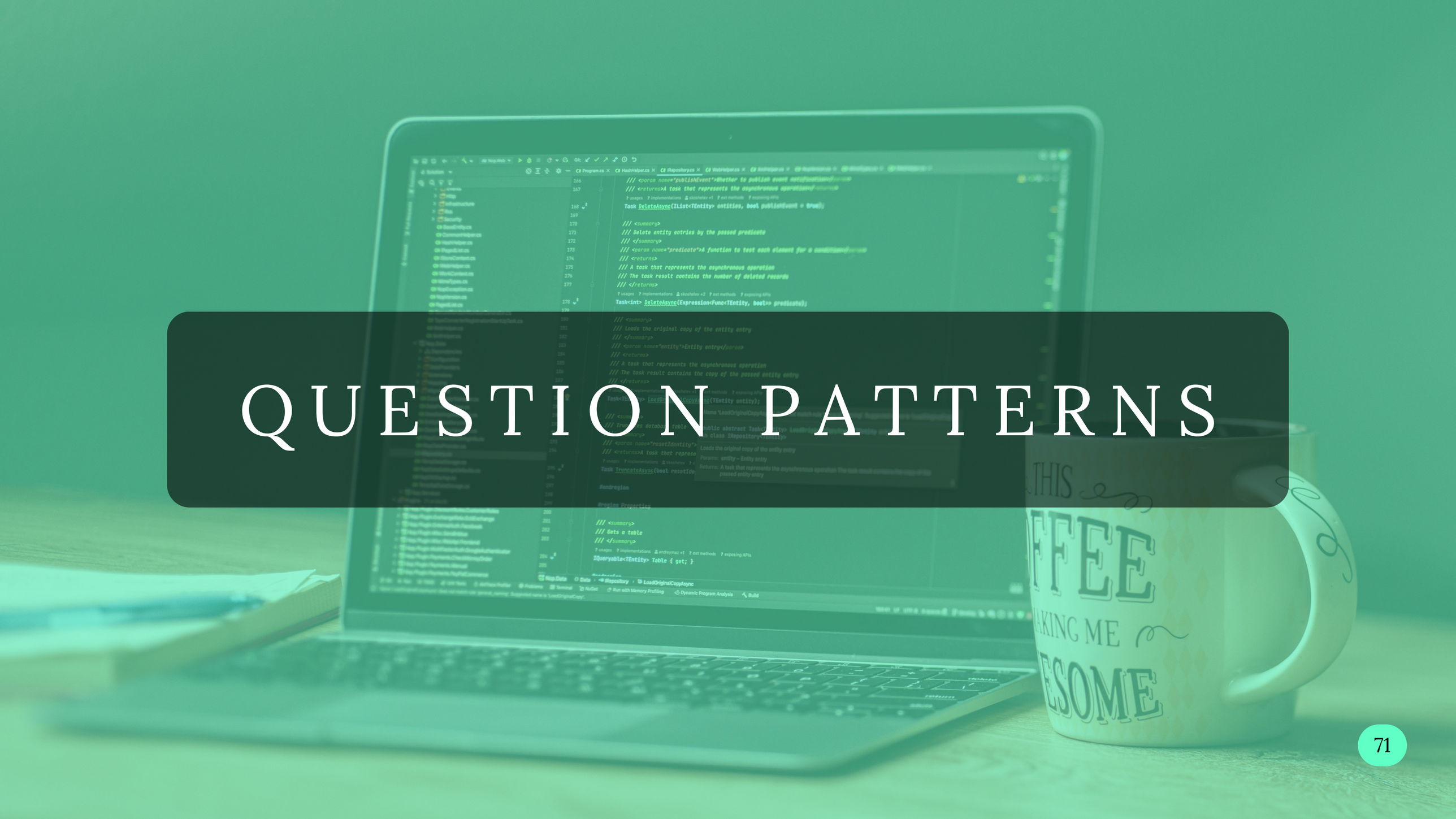
LEVEL-ORDER TRAVERSAL

Level-Order Traversal visits nodes level-by-level



Level-Order Traversal Sequence:

1 2 3 4 5



QUESTION PATTERNS

QUESTION PATTERNS

- **Write the Recursive Function Calls and Returns in a Tree Traversal Technique:** Given a Binary Tree, simulate Pre-Order, In-Order or Post-Order Traversal step-by-step. Write the recursive function calls and returns (as shown in the slides).

Note: Theory questions will come from Tree Theory, but none will require memorization

THE JOURNEY CONTINUES . . .